

Genesis of the HTTP/2 standard

As these changes took place, the nature of content rendered on the browsers also started to become more real-time as compared to static content that was rendered in the 1990s, right up till the mid-2000s.

To address this challenge, those at Google realised that browser behaviour needed to be addressed by providing additional information to it from the server, regarding which part of the content needed to be changed immediately and which portions had to remain static. The SPDY protocol, which was made public by Google in July 2012, was developed to provide information to the browser using a new command that would update only a certain part of the Web page with greater speed; hence, the protocol was called SPeeDY. It also used other techniques to speed up the transport of the files from the server to the browser by compressing the files using the gzip mechanism.

This was contributed by Google to the IETF standards body that took up the standardisation effort using the RFC 7540, which was completed in May 2015. Apart from other changes, a new protocol method, PATCH, was added. The PUT action replaces the data which is sent from the browser completely, in the server; with PATCH, the browser sends only the incremental changes that need to be done on the server and thereby enables quicker changes on the server side.

Early adopters

Google, with the SPDY protocol, started making changes in the behaviour of the HTTP server, and followed this by showing the performance improvements due to the Chrome browser enhancements to support this protocol. This grabbed the attention of many content delivery service providers who were handling high requests for the 'static' files that were hosted by them instead of application hosting servers. This was done predominantly to reduce the access times for these static files by hosting them nearer to the location of the browsers themselves.

Browsers that implemented HTTP/1.1 had the issue of locking up the requests thread to the server till the parse of the initial set of files was completed. This led to it taking longer to serve the browser's request for files and this affected the performance of these CDN (content delivery network) providers. This motivated Google and these adopters to make it a standard for the HTTP servers themselves, which led to the implementation of this version (from the draft stage itself) in the leading servers and browsers.



Figure 3: Browser support

Support for Web servers and browsers

Some of the popular Web servers and the corresponding versions that are supported are tabulated below:

Web server	Version supported
Apache Web Server	2.14.2 and above, using mod_h2
Nginx	1.9.5 onwards
Jetty://	9.3 onwards
Microsoft IIS	Windows 10

Browsers with the minimum version where the support is available are listed in Figure 3.

Many CDN providers, like Akamai (which spearheaded the adoption of this standard), CloudFlare and CDN77, have moved their servers to the latest versions that support this protocol. Apart from these providers, other application server provider who depend on high performance of HTTP servers also have moved to the latest versions.

The advantages of HTTP/2

Apart from compression of the data being sent from the server to the client, the following are additional changes that have been made in this version.

- A. *Concurrency support*: Multiple requests can be sent by the client over the same connection as opposed to a single request per HTTP connection in HTTP/1.1. The response can come in a different sequence than the order in which the requests were placed to quickly render the information that comes in early.
- B. *Support for prioritisation indication*: The browser can indicate to the server the priority of resources requested from it, and the server can change the sequence in which it transmits resources, thus helping the browser to handle these faster.
- C. *Server push support*: This helps in the server indicating the availability of additional information for the requests that have been made by the browser, which were not part of the request that it sent earlier.
- D. *Compression support*: Compression of data and the header reduces the time taken to transport the information to the browser.
- E. *PATCH support*: This is a modification of the PUT action, whereby the browser instructs the server to update the information partially instead of a complete replacement as in PUT.
- F. *RST_STREAM support*: This indicates to the server that the user has moved away from the operation that was requested (hit Back or stop the loading button, abort the download, etc), so that it can quickly abandon all the transfers from the server, instead of wasting bandwidth as happened with earlier versions.